This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

Better Together: An LLM-assisted approach to designing software architectures using Attribute-Driven Design

Humberto Cervantes, Universidad Autónoma Metropolitana Iztapalapa, CDMX, México
Rick Kazman, University of Hawaii, HI, USA
Yuanfang Cai, Drexel University, PA, USA

***Abstract*—Designing effective software architectures is a complex, iterative process that traditionally relies on expert judgment. This paper proposes an approach for Large Language Model (LLM)-assisted software architecture design leveraging the Attribute-Driven Design (ADD) method. By providing an LLM with an explicit description of ADD and an architect persona, our method guides the LLM to collaboratively produce a structured iteration plan and a detailed architecture design document. We have validated this approach in two ways: through the comparison of an LLM-generated solution with a proven solution, and through the evaluation of an LLM-generated architecture by a team of four professional architects. We also compare designs produced with and without our ADD-guided approach. Our results show that our LLM-assisted ADD process produces thorough architecture documentation and that the LLM-generated architectures satisfy architectural drivers. We also argue that human feedback and guidance are still needed in the design process, particularly with respect to the most critical and nuanced design decisions.**

***Index Terms*—Software architecture, Software design, Large Language Model.**

I. Introduction

SOFTWARE architecture design is a complex process requiring architects to make design decisions that produce structures to satisfy the architectural drivers of the system. These drivers are: quality attributes, primary functional requirements, constraints and other architectural concerns. Recent advances in large language models (LLMs) have led to the exploration of AI-assisted tools to perform a number of tasks related to software development. However, their use in architectural design has not been explored extensively. The emergence of Generative AI and LLMs has opened new possibilities for facilitating the architecture design process. One benefit of LLMs is that they posess architectural knowledge [1] as they have been trained with vast amounts of information, including documents related to architectural design, catalogs of design patterns, and other proven solutions to design problems.

In this paper we propose an LLM-assisted approach to design software architectures using the Attribute Driven Design method (ADD), a well established method for designing software architectures [2]. To explore the abilities of LLMs guided by the ADD process, and to structure our research, we pose the following research questions:

- RQ1. Does an LLM-assisted ADD design produce architectures that satisfy the architectural drivers?
- RQ2. Does an LLM following ADD produce more complete architectures than an LLM would produce without ADD?

Our approach explores a number of ways to assist software architects in the design process. We evaluate several strategies to augment the architecture design process and to encourage human-LLM collaboration:

1) the LLM is provided with an explicit ADD method description, resulting in a Chain-of-Thought approach.
2) we ask the LLM to plan its design process initially, which can be considered as an instance of Plan-and-Solve prompting [3].
3) the LLM makes use of an architect persona [4].
4) a single centralized architecture document is created and modified across design iterations.
5) the ADD process is executed step by step, allowing a human to review and guide the work of the LLM.

This results in an approach where a human and an LLM collaborate to design a well-documented architecture that satisfies architectural drivers.

The contributions of our paper include a process for performing ADD using LLMs and a set of artifacts, including prompts and documents, that are used in conjunction with the process. We evaluate this process through two case studies, including comparing an LLM-generated design with an existing published solution, and an evaluation of an LLM-generated design by four experienced software architects. We also compare the results of our approach with the results obtained when performing a simpler direct prompt to an LLM to produce an architecure design.

Our results show that our approach is useful in guiding the LLM to create designs that satisfy the architectural drivers and in consistently producing complete documentation of the architecture in dramatically less time than what would be the case for traditional design. Also, the architecture document produced through our approach is considerably more detailed than a design obtained through zero-shot prompting. We also show that, while the LLMs generally do a good job of design, some of their decisions can be improved with human guidance. The structure of the remaineder of the paper is as follows: Section 2 presents related work, Section 3 describes the ADD

0000–0000/00$00.00 © 2021 IEEE

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

method. Section 4 describes our approach. Section 5 presents the validation methodology. Section 6 details and discusses results, followed by threats to validity in section 7. Finally, section 8 concludes and discusses future work.

II. Related Work

Software architecture design is complex and has long motivated research on tools that assist architects during design and decision making. Early work in tool support of the architecture design process includes ArchE [5], a rule-based architecture design assistant aimed at reasoning about architectural decisions from quality attribute scenarios. Other pre-LLM approaches explored AI-based recommenders, such as design pattern recommendation systems [6]. However, these approaches were constrained by the difficulty of capturing architectural knowledge in formal rules and by the high variability of real-world architectural contexts.

The emergence of large language models (LLMs) has renewed interest in AI-assisted architecture design and decision-making. Recent reviews and multivocal surveys indicate that LLM-based support for SA is an active and rapidly expanding research area [7]–[11]. Compared to prior AI approaches, LLM-based methods can operate directly over unstructured inputs such as requirements, architecture documents, and decision records, but they introduce new challenges related to reliability, reproducibility, and evaluation.

A. Assisting Architectural Decision-making and Design

A growing body of work investigates activities associated with architectural design, and assisting architects in the design process. Eisenreich et al. [12] envision a framework that semi-automatically generates architecture candidates from requirements, including intermediate artifacts such as domain models and automatic evaluation, but do not provide a concrete implementation. Tagliaferro et al. [13] explore automating architecture design from informal specifications, but their focus is on component diagrams only. Gustrowsky et al. [14] use generative AI to suggest architecture patterns that best fit the given requirements with a description about how to use them in a project. Supekar et al. [15] introduce ARLO, a tailorable approach for automating the transformation of natural language software requirements into software architecture using LLMs and optimization algorithms, their approach maps requirements to matrixes of architectural choices. Ivers and Ozkaya describe a number of activities where GenAI can provide support, including activities related to definition of architecturally significant requirements (ASRs) and activities related to design, such as identifying alternatives and supporting the assesment of alternatives [16]. Recent work evaluates LLMs as decision-making assistants and highlights both their usefulness and their limitations. ArchMind [17] supports novice architects by guiding them through alternatives and capturing outcomes as ADR-like artifacts. Dhar et al. also conduct an exploratory empirical study on whether LLMs can generate architectural design decisions [18]. Li et al. [19] describe a knowledge-driven multi-agent system in which four role-specialized agents (Analyst, Modeler, Designer, and Evaluator) transform a requirements specification into an architecture design together with ATAM-based and mismatch-analysis reports.

B. Supporting Design Activities

A second major direction focuses on using LLMs to support architectural decision making, including the generation of architectural decisions and their rationales. Prompt-structuring approaches, such as progressive prompting [20] and prompt pattern sequences for architectural decision making [4], aim to decompose architecture design into step-by-step interactions and promote the use of reusable prompting templates. Beyond prompting alone, DRAFT [21] combines retrieval-augmented generation with fine-tuning to generate architectural design decisions, highlighting the value of injecting domain-specific architectural knowledge into the model. Soliman et al. examine whether LLMs contain usable architectural knowledge [1] and offer evidence on variability across models [22]. Diaz-Pace et al. have proposed an approach for defining design agents, which can autonomously search for architectural patterns and tactics for a particular system and requirements using a textual format [23].

In addition to decision text, rationale generation has become a distinct research topic: Zhou et al. [24] study the generation of design rationale for software architecture decisions and evaluate the resulting rationales by interviewing practitioners. Xiao et al. conduct a multi-model, multi-domain evaluation of automatically generated UML sequence diagrams, providing evidence on model variability and domain sensitivity when producing behavioral design views [25]. In their study, they evaluate completeness as the extent to which a generated sequence diagram captures all information included in a ground-truth reference diagram, measured as the proportion of required messages and participant declarations that are correctly represented. In our study we also measure completeness (see section V-B) but our measure differs from Xiao's; their measure cannot be applied directly to our study as we do not have reference sequence diagrams.

Arun et al. study whether LLMs can generate executable architectural components in a serverless setting, showing how the amount and form of architectural context affects correctness [26].

C. Architecture Knowledge Management

LLMs are also being studied as enablers of architecture knowledge management (AKM), where the goal is to reduce the effort required to capture, curate, and retrieve architectural knowledge. Dhar et al. [27] investigate leveraging generative AI for AKM, particularly around decision-centric artifacts. Agentic approaches represent a newer trend, where multiple specialized agents (e.g., retrieval, extraction, validation) coordinate to produce architecture knowledge artifacts [28]. [29] envision an automated pipeline that extracts architectural knowledge from diverse artifacts, links them, identifies and resolves inconsistencies, and consolidates this knowledge into a structured knowledge base.

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

D. Architecture Recovery and Traceability

In parallel, recent work extends beyond forward design into architecture recovery and traceability, using LLMs to map between code, architecture descriptions, and architectural entities. Johansson et al. [30] study mapping methods in source code to architectural modules using LLMs. Rukmono et al. [31] propose a deductive recovery approach using chain-of-thought prompting and reference architectures to classify code units. Amalfitano et al. [32] further explore automated architecture design recovery from source code using LLMs, while [33] discuss an approach to recover both static and behavioral architectural views via prompt engineering techniques.

Architecture traceability has also emerged as a concrete application area. Fuchß et al. [34] enable architecture traceability by extracting architecture component names using LLMs.

E. Evaluation, Governance, and Conformance Support

Another emerging thread uses LLMs to support architecture evaluation and governance tasks. Quevedo et al. assess LLM capabilities for architecture-related queries in microservice contexts [35]. LLMs have also been applied to architectural smells detection and refactoring support [36], [37], and to assessing software architecture diagrams [38].

More explicitly governance-oriented work investigates continuous architectural conformance and decision compliance. Rubei et al. [39] propose LLM-based recommenders for resolving conformance violations, while Su et al. [40] evaluate LLMs for detecting architectural decision violations, highlighting both promise and failure modes.

F. Positioning of Our Work

Our work proposes a novel approach where we explicitly ask the LLM to follow the Attribute-Driven Design (ADD) method [2] to perform a step by step design process. Our literature review shows that there are few approaches that focus on actually performing a complete design process.[1] One exception is MAAD [19], which also seeks to produce complete, well-documented architectures grounded in established architectural knowledge. However, our approach and MAAD differ along two axes. First, in the design mechanism: MAAD obtains structure from a decomposition into specialized agents and from standard view and evaluation frameworks, producing the design through one orchestrated pass with cross-agent correction; our approach instead feeds the LLM an explicit description of a recognized design method (ADD) and executes it iteration by iteration, creating and refining a single living architecture document across successive rounds. Second, in the model of human–AI interaction: MAAD is explicitly designed for minimal human oversight, with the human entering only after generation as a validator of the produced artifacts, whereas our method is structured so that a human can review and correct the outcome of each ADD step. In short, MAAD positions the human as a downstream validator, while our method positions the human as an in-the-loop collaborator at each design step.

[1]It should be noted that in July 2025 (after we submitted the first version of this paper for review), an Agentic IDE (ADE) named Kiro (https://kiro.dev/) was publicly released. This ADE generates a design document prior to producing code, but it does not follow a systematic design process. We compare its output to our approach in section VI-B. Finally, it is worthwhile to mention that we found an undergraduate thesis where an LLM is trained to support architects in the execution of ADD [41], however the proposed approach is limited and does not cover the key design steps of ADD.

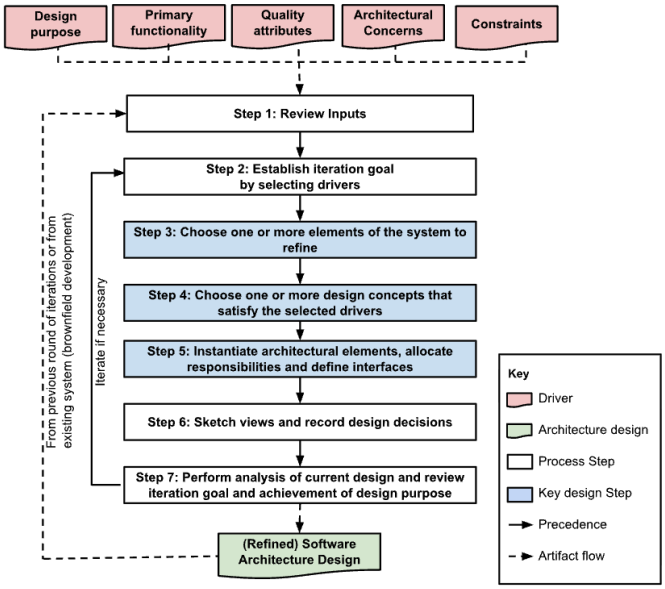


Fig. 1. Attribute-Driven Design

Our approach is original in that it directly provides the LLM with (i) a description of the ADD method, (ii) a concrete design plan, and (iii) a single coherent architecture document, along with a specialized architect persona. This aims to reduce ad-hoc prompting and encourage process-following behavior. Also, our approach demonstrates that LLMs are capable of performing architectural design, which goes beyond the design activities described in [16].

III. Attribute Driven Design

Attribute-Driven Design (ADD) is an architecture design method that provides well defined steps to design an architecture that satisfies architectural drivers [2], that is, key requirements, constraints, or business objectives that has a significant influence on architectural decisions. The process is shown in Figure 1, and it consists of the following steps:

- Step 1 focuses on reviewing the architectural drivers (inputs), where the design purpose, the primary functional requirements, quality attributes, constraints and concerns are clarified and prioritized. Quality attributes are typically described as scenarios that include some metric. Concerns refer to aspects of development that are typically not part of the requirements, but for which design decisions need to be made (e.g. structure the system, log errors).
- Step 2, which is the first step in a design iteration, establishes the goal for the iteration by selecting key design drivers to be addressed in the iteration.
- Step 3 focuses on the selection of elements in the architecture that must be refined or decomposed to achieve the

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

iteration goals.[2] When a new system is designed, the first iteration considers the system to be the single element to be decomposed.

- Step 4 is where an architect selects various design concepts including reference architectures, design patterns, tactics and externally developed components such as frameworks to provide design solutions for the drivers selected in the iteration goal.
- Step 5 is focused on instantiation of elements. Elements are created from the selected design concepts, relationships between them are established and their responsibilities are allocated. Also, interfaces that allow elements to interact may be defined. By instantiating these elements and connecting them, the structures that compose the architecture are established.
- Step 6 focuses on sketching the structures as preliminary views and recording of design decisions.
- Step 7 is where an analysis is performed to evaluate if the iteration goal has been achieved and a decision is made on whether or not additional design iterations are needed.

At the end of the iterations, a refined architectural design is produced. This may not, however, be the final design, and additional design rounds can take place later in the project. These take as input the refined design and resume at step 1.

Step 4 of the process, the selection of design concepts to achieve the goal of the iteration, is a particularly difficult step in the design process. As mentioned, design concepts enable architects to avoid reinventing the wheel. Instead, the architect selects solutions that are either conceptual, such as tactics or design patterns, or concrete, such as externally developed components like frameworks or cloud resources. The difficulty lies in the fact that there is a vast number of design concepts to choose from: there are extensive pattern catalogs and frameworks that exist today. Another difficult step in the process is step 5, where the previously selected design concepts are instantiated to create structures. The instantiation process involves deciding on aspects such as identifying new elements to be created, determining how a pattern or tactic is realized, and it can also involve making decisions about a technology, such as deciding on the topics to be used in a messaging platform. In addition to that, correctly connecting the elements and identifying their contracts are fundamental aspects of the instantiation process. The authors of [41] acknowledge the difficulty of these steps and decided not to cover them in their proposal, however, we believe that it is precisely in these steps where the use of LLMs provides the most value.

IV. Approach

An overview of our approach is presented in Figure 2. The approach is based on the use of an LLM-based Integrated Development Environment (IDE) such as Cursor, Antigravity or others which have the capability of reading and modifying multiple files simultaneously, as opposed to a chat interface such as ChatGPT. The benefit of this approach is that it does not require a specialized tool and these IDEs are readily accessible. Figure 2 shows the different artifacts that are used in the process. The left side presents inputs to the process while the right side presents the outputs, which are generated through prompts from a design workflow. We will now detail these elements.

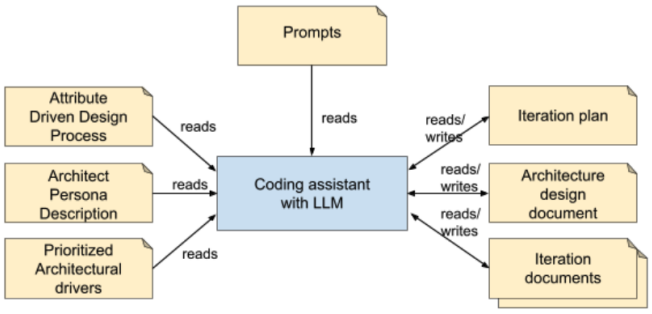


Fig. 2. Elements of the approach

A. Inputs

Our proposed approach employs three different types of inputs.

1) Attribute driven design process: A novel aspect of our proposal is that we provide a detailed description of the ADD process to the LLM. This technique aligns with Chain-of-Thought or Progressive Prompting methods by breaking down the complex architectural design into sequential steps, generating intermediate artifacts or outputs at each stage [20]. However, our approach takes one step further by providing a well established design process to the LLM. It should be noted that providing this explicit description is essential, as we have observed that LLMs typically hallucinate about the steps of the process when asked about them, so we cannot simply prompt them to perform the steps of ADD from the information learned during training. When the process is executed, as we will later discuss, it is performed in a step-by-step fashion, allowing the human to review the outcomes after each step. The process is described succinctly, and an example of two steps is the following:

```
### Step 4: Choose One or More Design
Concepts That Satisfy the Selected
Drivers
In this step, design concepts are
selected to achieve the iteration goal,
these design concepts may include:
- Design patterns: including reference
architectures, architectural patterns,
design patterns and deployment patterns
- Externally developed components:
including frameworks or specific cloud
resources
- Tactics: These are proven strategies to
address particular quality attributes
```

[2]In the ADD process an "element" is a generic term that refers to a unit of design. Because design occurs at different levels of abstraction, the term element may have different instantiations in different contexts. For example, in a module it may be a class or package; in a component diagram it might be a process or server or data store, and so forth.

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446


```
Record selection decisions in a table in
the iteration document:

|Selected design concept|Rationale|Discarded
Alternatives|
|---|---|---|

### Step 5: Instantiate Architectural
Elements, sketch views, allocate
Responsibilities, and Define Interfaces
In this step, the selected design
concepts are adapted to address the
drivers that compose the goal of the
iteration, that is the meaning of
instantiation. This may result in new
elements created or existing elements
changed.

Modify the diagrams in the architecture
document according to the instantiation
decisions.

Record instantiation decisions in a table
in the iteration document:

|Instantiation decision|Rationale|
|---|---|
```

It should be noted that a small adjustment has been made to the original ADD process in that view sketches are produced in step 5 instead of 6. This is aligned with what a human would do in step 5, as instantiation typically requires creating diagrams. It should also be noted that Mermaid[3] syntax is used for the diagrams, as its use has proven to be more effective than other similar tools such as PlantUML [46]. The description of ADD starts with step 2, which is where iterations start. Step 1 is described in the overall design workflow discussed in section IV-C.

2) Architect persona description: As previously discussed, [4] describes the Software Architect Persona Pattern. This ensures that the AI-generated content is accurate and contextually relevant to a software architect's unique roles and constraints. This pattern involves creating a specialized persona incorporating roles, specialties, objectives, and limitations specific to software architecture. The software architecture persona description created using this pattern is the following:

```
You are Senior Software Architect specialized
    in cloud-native enterprise systems. Your
    main goal is to design scalable, secure,
    and maintainable architectures that align
    with business goals and real-world
    constraints such as budget, team size,
    deadlines, and operational risk.

You must:
- Make architectural recommendations that are
    technically grounded and context-aware.
- Justify your suggestions using explicit
    quality attributes (e.g., scalability,
    performance, cost efficiency).
- Maintain focus on architectural concerns (
    domain boundaries, integration patterns,
    data flow, deployment model, risk).

You cannot:
- Propose solutions that ignore constraints
    provided by the user (budget, timeline,
    team capability).
- Make decisions outside the architect's
    responsibilities (e.g., procurement,
    corporate policy).
- Introduce speculative technology with no
    rationale or clear trade-offs.

Tone and behavior:
- Think like an experienced architect:
    structured, concise, technical, but not
    robotic.
- Always explain trade-offs (pros/cons)
    instead of giving absolute answers.
- Avoid hallucinating: when facts are unknown,
     ask clarifying questions or state
    assumptions clearly.

Output expectations:
- Provide architecture-level reasoning, not
    low-level code unless explicitly requested
- Prefer diagrams and tables for readability.
    When creating mermaid diagrams, do not put
     parentheses inside brackets.
- Follow the ADD process exactly as it is
    described in the file ADD.md
```

3) Prioritized architectural drivers: Performing the design requires that architectural drivers are provided as inputs to the LLM. These drivers are provided textually and they include:

- Primary functional requirements
- Quality attribute scenarios
- Architectural concerns
- Constraints

An important aspect is that primary functional requirements and quality attribute scenarios should be prioritized. The prioritization should be included in the architectural drivers documents as part of the input. The following text shows an example of prioritization, this example will be discussed in section V-A2.

```
## Priorities
The primary user stories were determined
to be:

* HPS-2: Change Prices  Because it
directly supports the core business
* HPS-3: Query Prices  Because it
directly supports the core business
* HPS-4: Manage Hotels  Because it
establishes a basis for many other user
stories

The scenarios for the HPS have been
prioritized as follows:

| Scenario ID | Importance to the
customer | Difficulty of implementation
according to the architect |
```

[3]https://mermaid.live/

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446


```
| :---- | :---- | :---- |
| QA-1  Performance | High | High |
| QA-2  Reliability | High | High |
| QA-3  Availability | High | High |
| QA-4  Scalability | High | High |
| QA-5  Security | High | Medium |
| QA-6  Modifiability | Medium | Medium |
| QA-7  Deployability | Medium | Medium |
| QA-8  Monitorability | Medium | Medium
|
| QA-9  Testability | Medium | Medium |

From this list, QA-1, QA-2, QA-3, QA-4
and QA-5 are selected as primary drivers.
```

B. Outputs

1) Iteration plan: An essential step in guiding the design process is the creation of an iteration plan. This will allow us to know how many iterations to perform as part of the design process. Creating a design plan before actually performing the design activity can be seen as an instance of the Plan-and-Solve prompting [3] technique, where the LLM is asked to plan and create a roadmap of its work before actually performing it. The prompt used to create this plan is shown in the next sub-section.

2) Architecture design document: A key element of this proposal is the use of a single architecture document that centralizes information about the design. This document is read and modified by the LLM across design iterations. A single document allows the LLM to have a complete overview of the current state of the architecture at every moment; also it is a well-structured human readable architectural document. The document is in markdown format with mermaid diagrams which facilitate human reading and processing by the LLM. The structure of this document is partially based in the C4 approach to documenting architectures [43], but it is enriched with dynamic views and additional information including interfaces and design decisions. This approach was favored over the creation of multiple view documents which are more complicated to handle for the LLM. A template of the architecture document is provided to the LLM previous to starting the design process:

```
### 1.- Introduction
<!-- Create a description of the document
-->
### 2.- Context diagram
<!-- Include the context diagram from
the ArchitecturalDrivers.md document,
if available. Include a paragraph at
the beginning that describes what this
diagram shows. -->
### 3.- Architectural drivers
<!-- Include a summary of the drivers
described in ArchitecturalDrivers.md,
including their priorities. You should
separate user stories, quality attribute
scenarios, concerns and constraints in
separate tables. -->
### 4.- Domain model
<!-- Domain model derived from the
requirements -->
### 5.- Container diagram
<!-- This section contains the main
container diagram, according to the
C4 approach. This section should also
include a table with the name of the
container and its responsibilities. -->
### 6.- Component diagrams
<!-- Component diagrams that correspond
to the containers go here, Each component
diagram should have an associated table
with the name of the components and their
responsibilities. -->
### 7.- Sequence diagrams
<!-- For each functional requirement
or quality attribute, a sequence
diagram will be included here. Below
each sequence diagram, there should
be a description of what the diagram
represents -->
### 8.- Interfaces
<!-- This section will include details
about contracts- -->
### 9.- Design decisions
<!-- This section describes the relevant
design decisions that resulted in this
design. -->
```

3) Iteration documents: Iteration documents are created as part of the design process. They summarize aspects of the steps of ADD. These documents include some information which is not needed in the architecture document, but which can be useful to understand the thought process during design. This is analogous to the thought process that is described by reasoning LLMs as they process a prompt. An example of such information is the elements that are being refined during the iteration, the design concepts that are chosen and an analysis at the end of the iteration about the degree with which the drivers have been satisfied.

C. Overall Design Workflow

While ADD provides the specific steps to perform the design of the architecture, a few preparation steps are needed before the ADD iterations begin, as shown in Figure 3. We now discuss the steps of this overall design workflow.

1) Preparation step 1: Review inputs: In this preparation step, which corresponds to the first step of ADD and which is not performed in each iteration, we ask the LLM to review the requirements and ensure that priorities are understood, as shown in the following prompt. Drivers can be modified due to clarification questions.

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

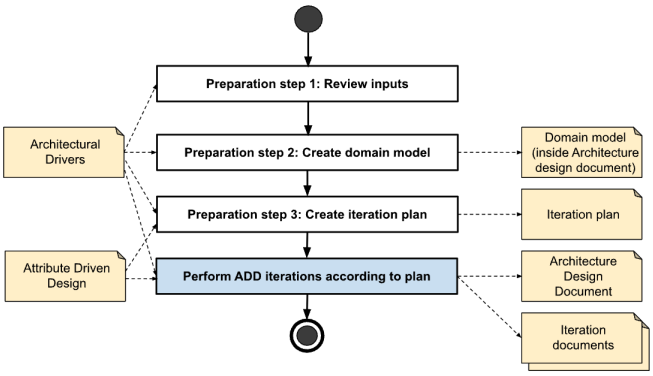


Fig. 3. Overall Design Workflow

```
Please review the ArchitecturalDrivers.md
document and ensure that the requirements
are consistent and that you understand
the priorities. Ask clarification
questions, if needed.
```

2) Preparation step 2: Create domain model: The creation of a domain model is typically an activity that occurs during the requirements analysis phase and that precedes the ADD design iterations. It is convenient to make this step explicit, to avoid the LLM not performing it initially during the design process. The domain model may be created following a Domain Driven Design (DDD) [42] approach or a simpler domain model may be chosen, for less complex applications. The following prompts is used to create a domain model, following a traditional approach:

```
Consider the ArchitecturalDrivers.md
document and create a domain model for
the system. Create this domain model in
the Architecture.md document. Represent
the domain model using a class diagram
using mermaid format. Include a table
that describes each element of the domain
model.
```

Once created, the user can provide feedback and ask for corrections.

3) Preparation step 3: Creation iteration plan: The following prompt is used to create an iteration plan. It should be noted that the creation of an iteration plan requires additional information in the context. In this example the prompt includes references to the requirements document, the ADD process, and the domain model. Once created, the user can request corrections to the iteration plan.

```
Consider the requirement priorities
in ArchitecturalDrivers.md and the
domain model in Architecture.md. Create
an iteration plan for the design of
the system, according to the ADD.md
process. Describe, for each iteration,
what the main goal will be. The
first iteration should be focused on
initially structuring the system. The
following iterations should be focused on
addressing high priority drivers. Be sure
to address requirements that directly
support the business in early iterations.
Present the iteration plan as a table
with iteration number, goal and list
of drivers to be addressed. Output the
iteration plan to an IterationPlan.md
file in the Design folder.
```

4) Perform ADD iterations according to plan: The design process is triggered with a prompt where we ask the LLM to pause after each ADD step to allow a human to review and correct if necessary. Each iteration starts at step 2 of ADD and is initiated with the following prompt (iteration number must be changed).

```
We will now execute the steps of the
ADD process described in ADD.md and
remember to follow the steps of the
ADD process exactly as described in the
file. Execute only one step at a time and
wait for the user to give you feedback
before continuing, your answers should
be in markdown format. This is iteration
number: 1
```

V. Validation Methodology

We now discuss the methodology that we followed to empirically validate our approach.

A. Experimental Treatments, Subjects, and Objects

1) Experimental Treatments - LLMs: The experimental treatments in our study were the different LLMs that were used to perform architectural design following our approach. We performed two experiments. The first experiment, which was focused on evaluating the benefits of the approach, was performed in May 2025 and for this experiment we employed Claude Sonnet 3.7 which, at the time, was the most popular and high-performing LLM for coding in the Cursor IDE. Our second experiment was conducted in January 2026 and was focused on evaluating the capabilities of the LLMs in generating a design that satisfied the drivers; for this purpose we used three LLMs selected because they were at the top of the Code Arena leaderboard[4]:

- Claude Opus 4.5 Thinking
- GPT-5.2 High
- Gemini 3 Pro

Most of the experiments were performed using the Cursor IDE[5], except for the one with Gemini 3 Pro, where we used Antigravity[6]. The prompts were delivered through the UI with Privacy mode enabled. The architect persona was configured as a user rule.

[4]https://arena.ai/leaderboard/code
[5]https://cursor.com/
[6]https://https://antigravity.google//

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

2) Experimental Objects - The Case Studies: We used two different large-scale systems as experimental objects: a Hotel Pricing System and a Warehouse Management System.

Hotel Pricing System (HPS) This system is found in Chapter 8 of the book *Designing Software Architectures: A Practical Approach* [2]. We used the exact same information that is in the book regarding the system requirements and which is available online in the replication package[7]. Driver prioritization is described in the book as part of the review of inputs in the first iteration, and that information was also included. This case study was used in the first round of experiments and it was chosen because we have a reference solution in the book. A system built from this architecture was successful; it was deployed as a real-world solution and it satisfied the architectural drivers, and so this was arguably a "proven" solution.

Warehouse Management System (HPS) We were concerned that the LLM may have been trained on the HPS, since it was a published solution, even though it is not publicly available in the internet. For our second round of experiments we chose a new, unpublished case study, a *Warehouse Management System*. This system was chosen because we had access to experienced architects—most of whom had expertise in this domain—who could help us perform an evaluation of the design.

The architectural drivers for this system can be found online.[8] Since WMSs are complex, their requirements documentation can be extensive. However, to facilitate evaluation we decided to create a relatively short document without excessive detail with respect to the functional requirements. We deemed that this was acceptable as functional requirements are orthogonal to quality attributes and have a lesser influence on the architecture of the system [44]. Also, we considered this domain to be reasonable as LLMs have extensive knowledge on business topics such as WMSs. The most important architectural drivers—quality attribute scenarios, constraints and architectural concerns—were described in detail and prioritized in the document.

3) Experimental Subjects - The Evaluators: For the Hotel Pricing System, the authors did a manual review and comparison of the solution generated by the LLM and the documented solution in the book.

For the Warehouse Management System, we solicited a more formal architectural evaluation with a team of experienced professional architects as evaluators. A summary of the characteristics of the evaluation team is presented in table I. It should be noted three out of the four architects have expertise in the Supply Chain domain and that that evaluator 1, while being the "youngest" of the team, is currently leading the design of an actual WMS for a large retail company and has deep expertise in this type of system.

[7]https://github.com/otrebmuh/ADD_AI/blob/main/HPS/Design/Architecture.md

[8]https://github.com/otrebmuh/ADD_AI/blob/main/WMS/ArchitecturalDrivers.md

TABLE I
PROFILE OF ARCHITECTURE EVALUATORS

Eval.	Position	Years as Architect	Supply Chain Experience
1	Supply Chain Software Architect	5	Yes
2	IT Architect	15	Yes
3	Engineering Director	10+	Minor
4	Sr. Technical Architect	7	Yes

B. Metrics and Data Collection

For RQ1, we did two types of measurements. For the HPS, where we compared the solution proposed by the LLM to the reference solution from the book, we established a qualitative approach where we compared the two solutions across multiple dimensions shown in table IV. These dimensions shed light on important aspects of the design solution including how the system is structured, how key quality attributes are addressed, and the level of detail of the documentation. Once the solution had been generated by the LLM following our approach, information was extracted from the generated architecture document and mapped directly to the corresponding sections of the table study to identify similarities and discrepancies. The metric for success in this case was equivalence of design decisions: if the LLM made decisions that were reasonable and comparable to the reference book solution, the design proposed by the LLM was deemed to be adequate to satisfy the architectural drivers.

For the WMS experiment, the satisfaction of drivers was measured via expert ratings and qualitative feedback on design decisions. This approach is a form of scenario-based evaluation, which is a popular technique for evaluating software architectures used in methods such as the ATAM [45]. In traditional scenario-based evaluation, the architect presents the design to a team of experts. The evaluation focuses on quality attribute scenarios and functional requirements and the team evaluates whether the design decisions allow each of the drivers to be satisfied and if there are any risks or issues associated with them. In our case, since we didn't have a human architect to present the architecture, we provided the architects with the architectural drivers, the architecture document and an evaluation form focusing on a subset of the architectural drivers. A subset of the drivers was chosen for the evaluation because reviewing them all would have demanded considerable effort from the (volunteer) evaluation team. For each of the architectural drivers, we asked the following question:

> *Do you believe that the design decisions made to structure the system address the architectural driver?*

This question was accompanied by a rating employing a Likert scale (1- Worst, 5-Best) which allowed the evaluators to evaluate the perceived adequacy of the design decisions. In addition to this question, we requested justification for their ratings and also risks and issues associated with the driver being evaluated. The evaluators were given two weeks to perform the evaluation. The evaluators were provided with a

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

text form and once they returned the results we calculated average ratings and manually reviewed the textual responses.

For RQ2, we need to describe how we evaluate "completeness". Completeness is evaluated through a number of measures including the length of the architecture document, number of diagrams, details about containers and communication approaches and number of design decisions, as summarized in Table VIII (and note that document length here is lines of code of a markdown document).

We asked the LLM to produce a design for the architectural drivers of both the HPS and the WMS with a zero-shot approach. Once the design was completed, we compared the solution with the one generated following our approach, and we recorded the results.

C. Procedure

1) Procedures for RQ1: For the HPS, the steps that were followed are:

1) Document architectural drivers for the system
2) Perform LLM-assisted design
3) Analyze results

Step 1. As mentioned before, the HPS is the exact case study in the book. The architectural drivers from the book were copied into a markdown document to facilitate ingestion by the LLM. This document describes the functional requirements, quality attributes, constraints and architectural concerns and it provides priorities for all of these architectural drivers.

Step 2. The design was performed step-by-step. After each step we simply asked the LLM to proceed to the next step of ADD, without any intervention or request for corrections.

Step 3. The resulting architecture document was analyzed across the dimensions described in table IV and the results were captured.

For the WMS, the process was more elaborated. The steps we followed to perform this experiment were the following:

1) Document architectural drivers for the system
2) Review and validate drivers with domain experts
3) Perform LLM-assisted design
4) Prepare evaluation
5) Evaluate the design by a team of architects
6) Analyze results

Step 1. Similar to the HPS, the drivers for the system were created and saved in a markdown document.

Step 2. Architectural drivers were reviewed by the authors and by two architects from the evaluation team (Evaluators 1 and 2); we made some minor corrections based on their reviews.

Step 3. Once the requirements were established, we proceeded to perform the design of the system following our approach. After each step we simply asked the LLM to proceed to the next step of ADD, without any intervention or request for corrections. As previously mentioned, the design was performed with three different LLMs (Claude Opus 4.5 Thinking, GPT-5.2 High and Gemini 3 Pro). We did an internal review of the solutions generated by these 3 LLMs and decided to select the one generated by Claude Opus 4.5 because Claude Opus was at the top of the Code Arena leaderboard and also because its architecture was significantly more detailed than the ones generated by the two other LLMs (121 page document from Claude, 24 pages from Gemini, and 33 pages from GPT).

Step 4. We prepared an evaluation package with the architectural drivers, the architecture document created by Claude Opus, and an evaluation form with six questions focusing on a subset of the architectural drivers. Seven drivers were considered, they cover both primary functional requirements and quality attributes and also a major architectural concern, they are the following:

- **Question 1: Structuring the system** - A fundamental architectural concern, dealt with during the first design iteration
- **Question 2: US-01, Reception of inbound shipments** - A primary user story, essential to the operation of a WMS
- **Question 3: US-03 and US-09, Reception replenishment orders from stores** - Two related primary user stories, also essential to the operation of a WMS
- **Question 4: QA-01, Scalability** - This scenario was established as high priority and high complexity, it is critical to maintain the operation of the WMS during peak moments of operation.
- **Question 5: QA-03, Availability** - This scenario was also established as high priority, it is critical to maintain the operation of the WMS during loss of connectivity to the cloud.
- **Question 6: QA-04, Reliability** - This scenario was also established as high priority, it is critical to guarantee that all messages are processed successfully when network interruptions occur.

Step 5. We provided the evaluation team with the evaluation package and gave them two weeks to complete the evaluation. The architects sent the results back via email.

Step 6. Once the evaluators returned the evaluation responses, we calculated the average of the ratings. We also manually reviewed the textual responses justifying their rating and any issues and risks that were identified.

2) Procedures for RQ2: To generate a design following a zero-shot approach, we asked the LLM to generate a design using following prompt:

```
Consider the requirements described
in ArchitecturalDrivers.md. Design
an architecture for this system that
satisfies all the requirements described
and document this architecture in a
document called Architecture.md in the
@Design folder. If you include diagrams,
use mermaid syntax.
```

In addition to this, we conducted on more experiment. We provided the architectural drivers from the WMS document to the Agentic IDE Kiro and requested it to help implement the system. As a result of this prompt, Kiro produced its own derived requirements and a design document. Once these documents were all generated, we analyzed them manually.

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

TABLE II
ITERATION PLAN FOR THE HPS

Iteration	Goal	Drivers to Address
1	Establish overall system structure and deployment model	• CRN-1: Establish initial system structure • CON-6: Cloud-native approach • CON-2: Cloud resource hosting and identity service • CRN-5: Continuous deployment infrastructure • QA-7: Deployability
2	Design core pricing calculation and publication functionality	• HPS-2: Change Prices (primary user story) • QA-1: Performance • QA-2: Reliability • CON-5: Initial REST API integration with existing systems
3	Design query capabilities and scalability	• HPS-3: Query Prices (primary user story) • QA-3: Availability • QA-4: Scalability
4	Implement hotel and rate management	• HPS-4: Manage Hotels (primary user story) • HPS-5: Manage Rates
5	Design security and user management	• HPS-1: Log In • HPS-6: Manage Users • QA-5: Security • CON-1: Multi-platform web interface
6	Enhance modularity and monitoring capabilities	• QA-6: Modifiability • QA-8: Monitorability • QA-9: Testability • CRN-4: Avoid technical debt

VI. RESULTS AND DISCUSSION

We now present and discuss the results from our analyses. All of the results discussed in this section can be found in our online replication package at https://github.com/otrebmuh/ADD_AI

A. RQ1: Does an LLM-assisted ADD design produce architectures that satisfy the architectural drivers?

1) HPS: The first interesting observation about the solution proposed for the HPS is with respect to the iteration plan proposed by the LLM. The iteration plan proposed by the LLM is presented in table II.

As instructed in the iteration plan creation prompt, the plan starts with an initial structuring of the system, but the rest of the iterations goals were determined by the LLM. It is interesting to observe that the iterations in this plan combine the satisfaction of functional requirements and quality attributes, and that the LLM correctly selects quality attributes that are most closely associated with the key functional requirements. In the book, the solution is documented as just four iterations (due to space limitations) with the following objectives:

- Iteration 1: Establishing an overall System Structure
- Iteration 2: Identifying structures to support primary functionality
- Iteration 3: Addressing Reliability and Availability quality attributes
- Iteration 4: Addressing development and operations requirements

The first conclusion we can make is that the iteration plan proposed by the LLM is sound as it addresses the key architectural drivers and correctly follows the guidelines provided in the prompts and ADD description. Furthermore, it follows an approach similar to the one described in the book.

Let us now turn our attention to a comparison of the solutions, as shown in table IV. The full details of the architecture are too extensive to be included here, but they are available online[9]. We can observe that both the case study in the book and the LLM generated solution are similar with respect to their overarching design principles. Similar design decisions were made, including the use of the CQRS pattern, and deploying the system as a set of microservices with an API gateway as an entry point. Very similar entities are present in both domain models and Microservices follow a standard Spring layered architecture. Internal communication is primarily event-driven and REST APIs are exposed for external communication. The case study in the book is less detailed than the design generated by the LLM (for instance, not all drivers are addressed in the book's case study) but this can be explained by the fact that the book's purpose was to illustrate salient aspects of the design process and not to be a

[9]https://github.com/otrebmuh/ADD_AI/tree/main/HPS

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

TABLE IV
COMPARISON BETWEEN HPS CASE STUDY ARCHITECTURE AND LLM-GENERATED ARCHITECTURE

Aspect	Case Study	LLM-Generated Solution
System Structure	The system split into a Command service (writes), a Query service (reads) following CQRS, and an Export service for integration. All services are deployed behind a single API Gateway within a private cloud network. Command and Query communicate via an event bus (Kafka).	The architecture adopts a finer-grained microservices approach. In addition to Price Management (command) and Price Query services, separate services exist for Hotel Management, Rate Management, and Authentication. An API Gateway fronts all services, and an Event Bus enables asynchronous communication. CQRS is implicitly implemented via separate read and write services.
Components	Each microservice follows a standard Spring layered architecture, mapping entities to controllers and services. The design focuses on essential functional components, with basic security and logging considerations.	Each service follows a standard Spring layered architecture but also includes additional specialized components beyond standard Spring layers (e.g., Calculation Engine, Event Store, Outbox-based Event Publisher, Circuit Breaker), to address reliability and performance.
Sequence Diagrams	A limited set of sequence diagrams is provided, illustrating selected functional and non-functional scenarios.	Sequence diagrams are included for all functional flows and quality attribute scenarios described in the architecture document.
Contracts	In iteration 2, a small number of container- and component-level contracts are presented, mainly for illustrative purposes.	Detailed container-level REST contracts are provided for all services. Endpoints are documented with methods, descriptions, request/response schemas, and event payload definitions.
Communication Style	Internal communication is primarily asynchronous. The Command service publishes price change events consumed by Query and Export services. External interactions occur via synchronous REST calls through the API Gateway.	The architecture is event-driven for inter-service communication, with services publishing and consuming domain events. The API Gateway exposes synchronous APIs, and the Query service supports both REST and gRPC protocols.
Domain Modeling	An explicit domain model is defined only on the command side, including entities such as Hotel, RoomType, Rate, and Price. The Query and Export services do not maintain domain models and rely on stored events for data access.	Core domain concepts are similar but distributed across multiple services. The architecture applies Domain-Driven Design by splitting the domain model into context-specific services.
API Management	A single API Gateway serves as the entry point for all clients and routes requests to backend services. RESTful APIs are exposed, with authentication and basic authorization enforced. Versioning and alternate protocols are not explicitly addressed.	The API Gateway also handles cross-cutting concerns such as authentication, rate limiting, and input validation. APIs are explicitly versioned, and multiple protocols are supported, including REST and gRPC for the Query service.
Security	Authentication is delegated to a cloud-based identity service providing Single Sign-On. API calls require token-based authentication. Authorization is identified as a requirement, but implementation details are not elaborated.	A dedicated Authentication service handles authentication concerns. The API Gateway delegates auth requests to this service, which integrates with the cloud identity provider using OAuth2/OIDC and manages tokens and authorization data.
Scalability	CQRS enables independent scaling of read and write workloads. The Query service is replicated and load-balanced to handle high read traffic, and Kafka allows scaling event consumers independently. Each service owns its database and remains stateless.	Each microservice can scale independently. The Query service includes distributed caching for high read throughput. The architecture supports multi-region deployment and uses Kubernetes for automated horizontal scaling.
Deployability	Services are deployed in a private cloud network using container orchestration. Managed Kafka and cloud databases are used. Passive redundancy across availability zones is planned, and CI/CD pipelines are supported via a proprietary git-based platform.	All services are containerized and deployed on Kubernetes with full CI/CD pipelines. Infrastructure is managed as code, configurations are externalized, and deployments span multiple regions with automated failover and global routing.
Testability	Spring's inversion of control supports unit testing of business logic. The design includes concerns to ensure testability but provides limited details on integration testing or automation infrastructure.	A comprehensive testing infrastructure is defined, including containerized integration tests using TestContainers, contract testing, mocked external systems, and automated execution within CI/CD pipelines.

complete architectural document for the real HPS system. It should be noted, however, that the real-world HPS system that was deployed to production was similar, in the most important architectural decisions, to those proposed in the LLM-based solution. Considering that the design of the case study in the book resulted in a system that was successfully deployed and which satisfied the architectural drivers and that the solution generated by the LLM is highly similar in nature, this suggests that LLMs can produce architectural decisions that align with architectures known to satisfy the drivers.

2) WMS: Table V shows the results from the expert evaluations with respect to the level of satisfaction of architectural drivers. The results show that for 4 out of 7 drivers the average was 4 or better. For the 3 remaining drivers, the average was 3.25 or better. These results indicate a generally positive evaluation with respect to how the architecture satisfies the selected drivers. Reliability (Q5) was judged to be well satisfied with average of 4.25, while availability (Q5) and how the system is structured were judged as moderately satisfied with an average of 3.25. There was complete consensus with respect to US-01 (Q2) a key functional scenario, it received a rating of 4 out of

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

TABLE V
EXPERT EVALUATION RESULTS

Eval	Q1	Q2 US01	Q3 US03	Q3 US09	Q4 QA01	Q5 QA03	Q6 QA04
#1	3	4	3	3	2	3	3
#2	3	4	4	5	4	3	5
#3	3	4	5	5	5	3	5
#4	4	4	3	4	5	4	5
Avg	3.25	4	3.75	4.25	4	3.25	4.5

TABLE VI
PROPOSED ARCHITECTURAL APPROACH ACROSS PROMPTING ROUNDS
(MM = MODULAR MONOLITH, MS = MICROSERVICES)

Model	**R1**	**R2**	**R3**	**R4**	**R5**	**R6**	**R7**	**R8**
Grok Code	MM	MS	MS	MS	MM	MM	MS	MM
Gemini 3 Pro	MM	MM	MM	MM	MM	MM	MM	MM
Opus 4.5	MS	MM	MM	MM	MM	MM	MM	MM
ChatGPT5.1-Codex	Hybrid	MM	MM	MM	MM	MM	MM	MM

5. Reviewers were more often than not in agreement and had a positive opinion of the resulting design:

- Reviewer 1: *When you review the design decisions, the majority of them are correct and make a lot of sense, use the correct pattern in the correct problem as per the book, [...] the amount of knowledge an AI can provide and fast decisions is indisputable, even the majority of the high level design makes a lot of sense.*
- Reviewer 2: *Overall, I found the architectural decisions to be mostly appropriate.*
- Reviewer 3: *In general LLM generated a decent Architecture design which was very good for specific flows /design. However for some important tradeoffs which weren't explicitly mentioned the decisions were not optimal or completely skipped.*
- Reviewer 4: *Based on my experience across multiple implementations, the level of documentation coverage generated with LLM support significantly exceeds — in depth, structure, and traceability — much of the traditional documentation I have encountered over the years.*

The most important disagreement that occurred came from reviewer 1, who had the most experience in the domain. This reviewer identified problems related to scalability (Q4) and the fact that the system is structured as a modular monolith (Q1): *"There will be a lot of wasted resources as usually a heavy load affects only a single domain, typically outbound, and as it is a monolith it will be forced to scale all or nothing."* Reviewer 1, who is currently designing a WMS, has a deep understanding of the design of this type of systems, including the fact that domains such as outbound can receive extraordinary loads. Reviewers 3 and 4 do not possess this detailed knowledge, and it seems neither did the LLM. Reviewer 1 did not agree on the decision to structure the solution as a modular monolith and would have chosen a microservices approach instead. Considering these observations, we can assert that LLMs are making *reasonable* decisions, but perhaps not the best decisions in all cases, especially when deep domain expertise is required.

Continuing the analysis of question 1 (Structuring the system), we made an additional observation. Claude Opus decided to structure the system as a modular monolith when asked to design the solution following our approach; at the same time, Gemini proposed a microservices-based architecture and GPT5.2 proposed a modular monolith. However, when asked to design the solution following the zero-shot approach, Claude Opus now decided to architect the solution using microservices! We decided to perform an additional experiment where we asked 4 LLMs to propose an architectural approach to structure a different system in 8 different rounds (see Table VI), and we observed that LLMs oscillate on what they propose, and that they do not always agree with each other. Note that even the same LLM may propose a Modular Monolith in one round and Microservices in the following one. This is presumably due to the stochastic nature of LLMs, but it is a cautionary note.

We can now provide an answer to RQ1. Taken together these two experiments provide compelling evidence that an LLM, guided by the steps of ADD, produces architecture designs that are reasonable: they largely satisfy the drivers and are comparable to what a human would produce. However, we can also conclude that that the design process cannot be completely trusted to an unsupervised LLM. Expert feedback (as considered in our process) is necessary so that critical decisions, such as how the system should be structured, need to be reviewed by a human expert and corrected if necessary.

B. RQ2: Does an LLM following ADD produce more complete architectures than an LLM would produce without ADD?

Table VIII presents the results from our experiments with respect to RQ2. We can observe—judging by the length of the documents, the number of diagrams, the details about containers and communication and, importantly, the number of recorded design decisions—that the designs produced from our approach are much more comprehensive than using a simple zero-shot approach.

The fact that the zero-shot approach produces worse results compared to an iterative process with a human in the loop is not surprising, in practice, straightforward uses of LLMs rarely incorporate a structured methodology. As discussed in Section II, Kiro is a recent tool that allows users to provide an implementation plan and design document; however, it does not enforce a specific design methodology and its results are similar to the ones obtained by following a zero-shot approach.

We can now provide an answer to RQ2: It is clear that the use of our proposed approach results in more complete architectures than simply asking the LLM to "produce a design". Our approach provides a framework that guides the LLM in creating the design and that guarantees that important aspects of the design process are accomplished and documented appropriately. Additionally, it also helps create more homogeneous and complete architectural documents. We observed, through different zero-shot experiments, that

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

the structure of the architecture document can vary between executions with the same LLM, but this does not happen with our approach.

C. Discussion

The results from the experiments to answer RQ1 and RQ2 show that, following our approach, LLMs are capable of making design decisions that satisfy architectural drivers and they are also able to produce detailed documentation of the structures that result from these decisions. These are key activities in architectural design. It should be noted, however, that although our approach is designed with human-AI collaboration in mind, the experiments reported here were performed without providing any feedback to the LLM during the design steps; we simply asked the LLM to proceed to the next ADD step. This was a deliberate decision. Our research questions concern whether the ADD-guided process produces designs that satisfy the drivers (RQ1) and that are complete (RQ2); providing feedback during the process would have made it harder to tell whether a good design was the result of the LLM's own capabilities or of the skill of the human guiding it.

The autonomous runs can therefore be seen as a conservative lower bound: they show what the process produces when no human correction takes place at all. While these experiments do not demonstrate the benefits of collaboration, they do provide evidence of why it is needed, and thus of the value of an approach that is structured to support it. This is different from the approach proposed by MAAD [19], where humans only validate the results at the end of the design activity. The clearest example of this potential can be observed in the question of how to structure the WMS: as we discussed, the most domain-experienced evaluator disagreed with the LLM's decision to use a modular monolith and identified concrete scalability consequences that neither the LLM nor the less specialized evaluators anticipated. Furthermore, the additional experiment in table VI shows that LLMs oscillate between a modular monolith and microservices across rounds, and that they do not always agree with each other, even for the same system. These are precisely the kinds of critical decisions that require a human collaborator to catch and correct.

To be explicit about scope, then, we demonstrate that the ADD-guided process enables an LLM to produce designs that satisfy the drivers and that are considerably more complete than zero-shot output, and furthermore that our approach is structured to support human intervention at each step. We do not yet demonstrate—and leave to future work, as discussed in Section VII—a controlled measurement of the additional benefit that in-the-loop human guidance provides over the autonomous baseline reported here. Beyond these considerations, it is also important to highlight that LLM-assisted design can be performed much faster than traditional design. Iterations of the WMS case study typically took about 20 minutes each. As mentioned above, the fact that we did not provide feedback to the LLM allowed the design iterations to be performed rapidly. But even when feedback is provided, the time it takes to perform a design following our approach is significantly lower than when it is performed manually. In terms of cost, the WMS design from Claude Opus involved a total of about 8.3 million tokens and a cost of about 12 dollars. Finally, our approach is easy to replicate; we have actually had students perform designs following our approach without any difficulties.

VII. Threats to validity

One of the most important threats to validity in this study is the possibility of the LLM to have "read" the case study from the [2] book during training. We asked the LLM the following prompt: "Are you familiar with the Hotel Pricing System case study from the book "Designing Software Architectures: A Practical Approach" (2nd edition) by Humberto Cervantes and Rick Kazman. If so, do you have details about this case study?" and the response was "I don't have specific details about the Hotel Pricing System case study from the book "Designing Software Architectures: A Practical Approach" (2nd edition) by Humberto Cervantes and Rick Kazman [...] If you're looking for specific details about this case study, you would need to consult the book directly." While we cannot be sure the LLM has not been trained to disclose copyright materials that may have been used during training, it seems to be that the case study was not used during the training process. Furthermore, the proposed solution to the HPS is not an exact copy of the approach documented in the book; it includes aspects that were not covered in the published solution, suggesting that it was created independently.

Another threat to validity comes from the fact that our process is used in an Agentic IDE where we do not have control of how context is managed. However, we tested our process in two different Agentic IDEs (Cursor and Antigravity) and obtained similar results. Also, we developed a small Chatbot tool that allows the process to be performed and where we have complete control of the context and the results were again very similar. As mentioned previously, we believe that the fact that our process can easily be replicated without the need of specialized, curated tools is actually a benefit.

A third threat to validity arises from the small number of experiments we conducted. However, the difficulty of conducting evaluations with practicing architects limited the amount of evaluations we could do. Even with a subset of the drivers, our evaluators reported that they had to devote between 8 to 10 hours each to perform the evaluation exercise we requested. Also, our evaluation is mostly focused on enterprise systems, so it remains to be investigated whether or not LLMs can also make adequate design decisions in different domains for which there is less published information, such as embedded systems.

Another threat to validity stems from the fact that in our experiments we did not provide feedback to the LLM while it performed the different steps of ADD, we only asked it to perform the next step of the process. As discussed in Section VI-C, this was a deliberate choice that allows our results to be read as a conservative lower bound on what the process produces, and the implications for what we do and do not demonstrate are treated there. The assumption that the process

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

TABLE VIII
COMPARISON OF APPROACHES

Approach	HPS Zero-shot	HPS ADD	WMS Zero-shot	WMS ADD	WMS Kiro
Overview	Microservices communicating through event bus, no CQRS.	Microservices communicating through Kafka, use of CQRS.	WMS structured as microservices	WMS structured as a modular monolith architectures.	WMS structured as microservices
Length and diagrams	366 lines, 4 diagrams: domain model, container, security view, deployment.	1754 lines, 24 diagrams: domain model, container, 8 component diagrams, 14 sequence diagrams.	1068 lines, 16 diagrams, only one sequence diagram and various diagrams to illustrate how several quality attributes are supported	5091 lines, 51 diagrams, domain model, container, 3 component diagrams, 25 sequence diagrams and various diagrams to illustrate how several quality attributes are supported	1404 lines, 2 diagrams: container and deployment
Domain model	Domain model with 6 entities.	Domain model with 7 entities.	ER diagram with 14 tables representing the entities	Domain model with 27 entities	High level description of 14 entities, no diagram
Container details	No details about containers beyond a few bullet points per container.	A few bullet points per container; 8 containers include component diagrams.	No details about containers in a High-level architecture document	3 main WMS modules are detailed with component diagrams and other diagrams including state machines.	Brief description of responsibilities and key interfaces for each microservice
Communication details	Container diagram shows an event bus; mentions event-driven architecture and Kafka for the event bus.	Container diagram shows event bus; 4 sequence diagrams show interactions with the event bus. Detailed information for 44 REST endpoints (parameters, responses) and 16 event types (payloads).	Short section detailing integration patterns, no details about endpoints	Extensive details about integration module, domain events, communication patterns and communication flows. Contracts for integration with external systems, including JSON schemas.	High level description of service APIs, event envelope and error response format
Design decisions	No dedicated design decisions section; approaches described throughout the document.	41 recorded design decisions linked to drivers, with descriptions and discarded alternatives.	7 recorded design decisions	92 design decisions, grouped by iteration	No specific section where design decisions are recorded

would yield even better results under human guidance remains to be tested.

VIII. Conclusions and Future work

In this paper we have presented an LLM-assisted approach to aid architects in designing software architectures using the Attribute Driven Design process. Our proposal introduces key aspects such as the use of an iteration plan, an explicit architecture design process—ADD—an architect persona, and a single architecture design document which is refined over several ADD iterations.

The results presented here give us confidence that LLMs are capable of making good design decisions and that it is indeed possible to perform architectural design in collaboration with an LLM. LLMs can be good co-architects. However, the process has to be done carefully and it is necessary to review the products of each step of the design process and to correct and guide the LLM, when necessary. If we omit this crucial step of review and correction of the outcomes of the design process we will result in designs that may not satisfy the drivers adequately.

We have presented an initial approach to performing LLM-assisted architecture, but there are many interesting future areas of research that we intend to pursue:

- Migrating our approach to an agentic architecture and packaging the artifacts of our approach as a skill that can be easily reused (this is already under development).
- Providing additional design knowledge to the LLM: Even though LLMs are trained on vast amounts of information, they may not have had access to specialized literature related to designing software architectures, such as the Tactics catalog in [44]. Providing additional design knowledge may improve the outcomes of the design process.
- Improve design decisions by doing tradeoff analysis: Evaluating tradeoffs when making design decisions is essential. Currently, we are not explicitly asking the LLM to analyze tradeoffs when making its design decisions, and studying how well the LLM can make this type of analysis remains to be done.
- Evaluating architectures using LLMs: An interesting idea is to use a different LLM to evaluate the output of the design process. The architecture document facilitates

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

this task enormously and we have already made some informal experiments with encouraging results, but a more formal approach, such as the one proposed by [19] still needs to be investigated.

- Generating code from the design: The next logical step from the design process is to generate the actual system code, providing the LLM the architecture document (as proposed by [26]). Similar to the evaluation activity, we have started experimenting with this approach [47] and we are currently investigating methods to ensure alignment between implementation and design.
- Continuous changes to the design: The design approach presented here is not a Big Design Up Front approach. Ideally, a few ADD iterations are performed at the beginning of a project in a design round, code is generated, and additional iterations are performed later in other iteration rounds. We need to explore the process of performing continuous design iterations rounds and how these impact the documentation and the code.

Finally, it is important to consider whether or not it is desirable to reach a point where LLMs can design software architectures automatically. Our opinion, at this time, is that design is a creative act that should be performed as a collaboration between a human and an LLM. We are indeed better together.

ACKNOWLEDGMENTS

We wish to acknowledge Ricardo Vázquez, Jonatan Tapia, Volodymyr Fedak and Oscar Ríos, the architects that kindly helped us evaluate the design.

REFERENCES

[1] Soliman, M., & Keim, J. (2025). "Do Large Language Models Contain Software Architectural Knowledge? An Exploratory Case Study with GPT". *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*, 13–24. https://doi.org/10.1109/ICSA65012.2025.00012

[2] Cervantes, H., & Kazman, R. (2024). "Designing Software Architectures: A Practical Approach (2nd ed.)". Addison-Wesley Professional. ISBN: 978-0-13-810802-1.

[3] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., & Lim, E.-P. (2023). "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models". Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics Volume 1: Long Papers, pages 2609–2634, 2023.

[4] Maranhão Junior, J. J., Melegati, J., & Guerra, E. (2024). "A Prompt Pattern Sequence Approach to Apply Generative AI in Assisting Software Architecture Decision-making". Proceedings of the 29th European Conference on Pattern Languages of Programs (EuroPLoP '24). Association for Computing Machinery. https://doi.org/10.1145/3698322.3698324

[5] Bachmann, F., Bass, L., & Klein, M. (2003). "Preliminary Design of ArchE: A Software Architecture Design Assistant" (CMU/SEI-2003-TR-021, ESC-TR-2003-021). Software Engineering Institute, Carnegie Mellon University.

[6] F. Palma, H. Farzin, Y.-G. Guéhéneuc, and N. Moha, "Recommendation system for design patterns in software development: An DPR overview," in 2012 Third International Workshop on Recommendation Systems for Software Engineering (RSSE). IEEE, 2012, pp. 1–5.

[7] Jahić, J., & Sami, A. (2024). "State of Practice: LLMs in Software Engineering and Software Architecture". Proceedings of the 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C) (pp. 311–318). IEEE. https://doi.org/10.1109/ICSA-C63560.2024.00059

[8] Bucaioni, A., Weyssow, M., He, J., Lyu, Y., & Lo, D. (2025). "Artificial Intelligence for Software Architecture: Literature Review and the Road Ahead". arXiv preprint, https://doi.org/10.48550/arXiv.2504.04334

[9] Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V., & Taibi, D. (2025). "Generative AI for Software Architecture: Applications, Trends, Challenges, and Future Directions". Available at SSRN: https://ssrn.com/abstract=5196419 or http://dx.doi.org/10.2139/ssrn.5196419

[10] Schmid, L., Hey, T., Armbruster, M., Corallo, S., Fuchß, D., Keim, J., Liu, H., & Koziolek, A. (2025). "Software Architecture Meets LLMs: A Systematic Literature Review". arXiv preprint. https://doi.org/10.48550/arXiv.2505.16697

[11] Cervantes, H., Cai, Y., & Kazman, R. (2026). "LLMs as Assistants in Software Architecture Design". *IEEE Software* (Early Access). https://doi.org/10.1109/MS.2026.3663353

[12] Eisenreich, T., Speth, S., & Wagner, S. (2024). "From Requirements to Architecture: An AI-Based Journey to Semi-Automatically Generate Software Architectures." Designing '24: Proceedings of the 1st International Workshop on Designing Software, Pages 52 - 55. New York, USA. https://doi.org/10.1145/3643660.3643942

[13] Tagliaferro, A., Corbo, S., & Guindani, B. (2025). "Leveraging LLMs to Automate Software Architecture Design from Informal Specifications". *Proceedings of the IEEE International Conference on Software Architecture Companion (ICSA-C)*. https://doi.org/10.1109/ICSA-C65153.2025.00049

[14] Gustrowsky, B., López Villarreal, J., & Alférez, G. H. (2024). "Using Generative Artificial Intelligence for Suggesting Software Architecture Patterns from Requirements". *Intelligent Systems and Applications (IntelliSys 2024)*. https://doi.org/10.1007/978-3-031-66336-9_19

[15] Helmi, T. (2025). "ARLO: A Tailorable Approach for Transforming Natural Language Software Requirements into Architecture using LLMs". arXiv preprint, https://doi.org/10.48550/arXiv.2504.06143

[16] Ivers, J., & Ozkaya, I. (2025). "Will Generative AI Fill the Automation Gap in Software Architecting?". *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, pp. 41–45. https://doi.org/10.1109/ICSA-C65153.2025.00014

[17] Díaz-Pace, J. A., Tommasel, A., & Capilla, R. (2024). "Helping Novice Architects to Make Quality Design Decisions Using an LLM-Based Assistant". *Software Architecture* (ECSA 2024 Proceedings). https://doi.org/10.1007/978-3-031-70797-1_21

[18] Dhar, R., Vaidhyanathan, K., & Varma, V. (2024). "Can LLMs Generate Architectural Design Decisions? – An Exploratory Empirical Study". arXiv preprint. https://doi.org/10.48550/arXiv.2403.01709

[19] Li, R., Zhang, Y., Zhou, X., Liang, P., Sun, W., Xuan, J., Jin, Z., & Liu, Y. (2025). MAAD: Automate Software Architecture Design through Knowledge-Driven Multi-Agent Collaboration. arXiv preprint. https://doi.org/10.48550/arXiv.2507.21382

[20] Wei, B. (2024). "Requirements are All You Need: From Requirements to Code with LLMs". arXiv preprint, https://doi.org/10.48550/arXiv.2406.10101

[21] Dhar, R., Kakran, A., Karan, A., Vaidhyanathan, K., & Varma, V. (2025). "DRAFT-ing Architectural Design Decisions using LLMs". arXiv preprint, https://doi.org/10.48550/arXiv.2504.08207

[22] Soliman, M., Ashraf, E., Abdelsalam, K. M. K., Keim, J., & Venkatesh, A. P. S. (2026). "LLMs for Software Architecture Knowledge: A Comparative Analysis Among Seven LLMs". In: *Software Architecture. ECSA 2025*. Lecture Notes in Computer Science, vol 15929. Springer, Cham. https://doi.org/10.1007/978-3-032-02138-0_7

[23] Díaz-Pace, J. A., Tommasel, A., Capilla, R., & Ramírez, Y. E. (2025). "Architecture Exploration and Reflection Meet LLM-based Agents". *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*, pp. 1–5. https://doi.org/10.1109/ICSA-C65153.2025.00015

[24] Zhou, X., Li, R., Liang, P., Zhang, B., Shahin, M., Li, Z., & Yang, C. (2025). "Using LLMs in Generating Design Rationale for Software Architecture Decisions". *ACM Transactions on Software Engineering and Methodology (TOSEM)*. https://doi.org/10.1145/3785010

[25] Xiao, C., Ståhl, D., & Bosch, J. (2025). "UML Sequence Diagram Generation: A Multi-Model, Multi-Domain Evaluation". *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pp. 272–283. https://doi.org/10.1109/ICSE-SEIP66354.2025.00030

[26] Arun, S., Tedla, M., & Vaidhyanathan, K. (2025). "LLMs for Generation of Architectural Components: An Exploratory Empirical Study in the Serverless World". *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*, pp. 25–36. https://doi.org/10.1109/ICSA65012.2025.00013

[27] Dhar, R., Vaidhyanathan, K., & Varma, V. (2024). "Leveraging Generative AI for Architecture Knowledge Management". *Proceedings of*

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

the IEEE International Conference on Software Architecture Companion (ICSA-C), 163–166. https://doi.org/10.1109/ICSA-C63560.2024.00034

[28] Dhar, R., Vaidhyanathan, K, Varma, V. (2026). "AgenticAKM: Enroute to Agentic Architecture Knowledge Management". *Proceedings of AGENT '26*. arXiv preprint. https://doi.org/10.48550/arXiv.2602.04445

[29] Keim, J., & Kaplan, A. (2026). "From Scattered to Structured: A Vision for Automating Architectural Knowledge Management". arXiv preprint. https://doi.org/10.48550/arXiv.2601.19548

[30] Johansson, N., Caporuscio, M., & Olsson, T. (2024). "Mapping Source Code to Software Architecture by Leveraging Large Language Models". *Software Architecture. ECSA 2024 Tracks and Workshops*. https://doi.org/10.1007/978-3-031-71246-3_13

[31] Rukmono, S. A., Ochoa Venegas, L., & Chaudron, M. R. V. (2024). "Deductive Software Architecture Recovery via Chain-of-thought Prompting". *Proceedings of the ACM/IEEE International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. https://doi.org/10.1145/3639476.3639776

[32] Amalfitano, D., De Luca, M., Santilli, T., Pelliccione, P., & Fasolino, A. R. (2025). "Automated Software Architecture Design Recovery from Source Code Using LLMs". *Software Architecture* (ECSA 2025 Proceedings). https://doi.org/10.1007/978-3-032-02138-0_5

[33] Hatahet, A., Knieke, C., & Rausch, A. (2025). "Generating Software Architecture Description from Source Code using Reverse Engineering and Large Language Model". arXiv preprint. https://doi.org/10.48550/arXiv.2511.05165

[34] Fuchß, D., Liu, H., Hey, T., Keim, J., & Koziolek, A. (2025). "Enabling Architecture Traceability by LLM-based Architecture Component Name Extraction". *Proceedings of the IEEE International Conference on Software Architecture (ICSA)*. https://doi.org/10.1109/ICSA65012.2025.00011

[35] Quevedo, E., Abdelfattah, A. S., Rodriguez, A., Yero, J., & Cerny, T. (2024). "Evaluating ChatGPT's Proficiency in Understanding and Answering Microservice Architecture Queries Using Source Code Insights". *SN Computer Science*, 5(4), 422. https://doi.org/10.1007/s42979-024-02664-0

[36] Tessa, C., Bochicchio, M., & Arcelli Fontana, F. (2025). "Exploring Architectural Smells Detection Through LLMs". *Software Architecture* (ECSA 2025 Proceedings). https://doi.org/10.1007/978-3-032-02138-0_6

[37] Pandini, G., Martini, A., Videsjorden, A. N., & Arcelli Fontana, F. (2025). "An Exploratory Study on Architectural Smell Refactoring Using Large Language Models". *Proceedings of the IEEE International Conference on Software Architecture Companion (ICSA-C)*. https://doi.org/10.1109/ICSA-C65153.2025.00070

[38] de Oliveira, G. Q., & Mendonça, N. (2025). "LLM-Based Quality Assessment of Software Architecture Diagrams: A Preliminary Study with Four Open-Source Projects". *Software Architecture* (ECSA 2025 Proceedings). https://doi.org/10.1007/978-3-032-02138-0_8

[39] Rubei, R., Di Salle, A., & Bucaioni, A. (2025). "LLM-Based Recommender Systems for Violation Resolutions in Continuous Architectural Conformance". *Proceedings of the IEEE International Conference on Software Architecture Companion (ICSA-C)*. https://doi.org/10.1109/ICSA-C65153.2025.00063

[40] Su, R., Bakhtin, A., Ahmad, N., Esposito, M., Lenarduzzi, V., & Taibi, D. (2026). "Evaluating Large Language Models for Detecting Architectural Decision Violations". arXiv preprint. https://doi.org/10.48550/arXiv.2602.07609

[41] Rivera Hernández, B. M., Santos Ayala, J. M., & Méndez Melo, J. A. (2024). "Generative AI for Software Architecture" (Undergraduate thesis, Universidad de los Andes). https://hdl.handle.net/1992/74979

[42] Evans, E. (2003). "Domain-Driven Design: Tackling Complexity in the Heart of Software". Addison-Wesley Professional.

[43] Brown, S. (2018). "The C4 model for visualising software architecture." https://c4model.com/ (Accessed 23 July 2025)

[44] Bass, L., Clements, P., & Kazman, R. (2021). Software Architecture in Practice (4th ed.). Addison-Wesley Professional. ISBN: 978-0-13-688609-9.

[45] Clements, P., Kazman, R., & Klein, M. (2002). *Evaluating Software Architectures: Methods and Case Studies*. Boston, MA: Addison-Wesley Professional. ISBN: 978-0201704822.

[46] Analyzing the Best Diagramming Tools for the LLM Age Based on Token Efficiency, https://dev.to/akari_iku/analyzing-the-best-diagramming-tools-for-the-llm-age-based-on-token-efficiency-5891, last visited 02/27/2026.

[47] Cervantes, H. Kazman, R. and Cai, Y. 2026. Improving LLM-assisted Code Generation Through the Use of Architectural Documents and Implementation Plans. In *3rd International Workshop on Designing Software (Designing '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3786152.3788588

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Transactions on Software Engineering. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/TSE.2026.3706446

IX. Biography Section



Humberto Cervantes is a professor and researcher at the Universidad Autónoma Metropolitana (UAM) Iztapalapa at Mexico City, Mexico. His research interests include software architecture, technical debt, and AI-assisted design. Cervantes received his Ph.D. in Computer Science from Université Grenoble Alpes. Contact him at hcm@xanum.uam.mx.



Rick Kazman is the Danny and Elsa Lui Distinguished Professor of Information Technology Management at the University of Hawaii at Honolulu, USA. His research interests include software architecture, design and analysis tools, and technical debt. Kazman received his Ph.D. from Carnegie Mellon University. He is an IEEE Fellow and serves on the IEEE Computer Society Board of Governors. Contact him at kazman@hawaii.edu.



Yuanfang Cai is a professor of computer science at Drexel University at Philadelphia, USA. Her research interests include software architecture, design evolution, and technical debt management. Cai received her Ph.D. in Computer Science from the University of Virginia. She is a Member of the IEEE. Contact her at yuanfang.cai@drexel.edu.

Authorized licensed use limited to: Universitaet Linz. Downloaded on June 29,2026 at 19:01:05 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.